

THREE

Debugging Techniques

A considered approach to resolving complex software defects through system understanding, fresh perspectives, and fundamental verifications.

Techniques for Resolution

Understand the System

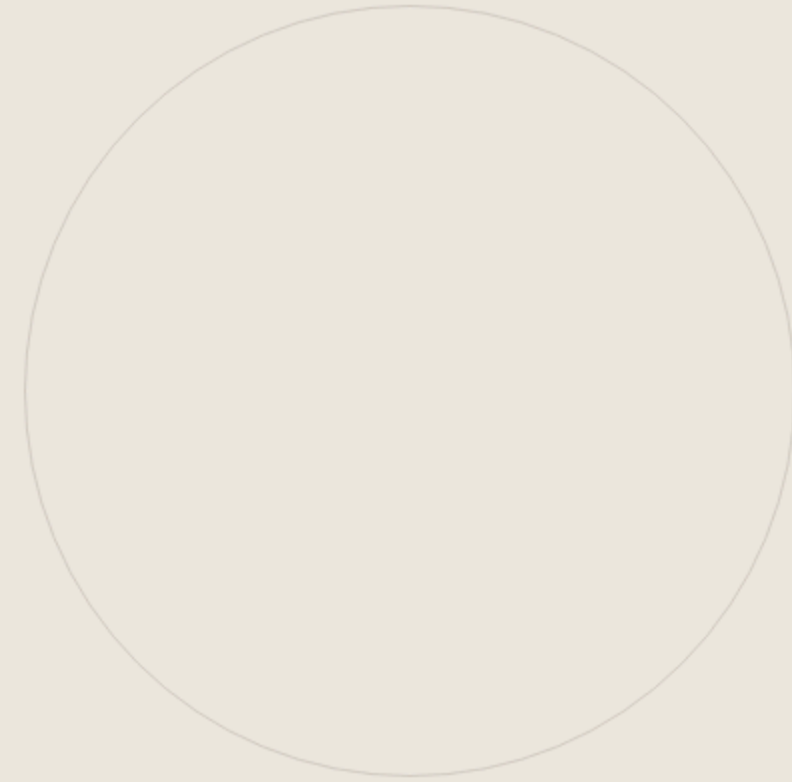
- 01 Reading the documentation and mapping the flow before changing code.
-

Get a Fresh View

- 02 Rubber duck debugging and walking away from the problem.
-

Check the Plug

- 03 Questioning fundamental assumptions and environmental setup.
-



Understand the System

Why use this technique?

Guessing at the cause of a bug often leads to "voodoo programming" or masking the symptom rather than curing the disease.

By thoroughly understanding the system's architecture, dependencies, and intended behavior, you can deduce where the failure must be occurring.

Before: Blindly guessing

```
// "I don't know why it's null, I'll just add a check"
if (user != null && user.Profile != null)
{
    RenderProfile(user);
}
else
{
    // silently fail
}
```

After: Understanding the data flow

```
// Traced to the source: The API now returns an incomplete
// object during the 'Pending' state. We must handle the state.
if (user.State == UserState.Pending)
{
    RenderLoadingSpinner();
}
else
```

Pros & Cons

Pros

- Leads to robust, long-term fixes rather than brittle patches.
- Increases your domain knowledge and architectural grasp.

Cons

- Can be time-consuming, especially in large legacy codebases.
- Requires up-to-date documentation, which is often lacking.

Get a Fresh View

Why use this technique?

Cognitive tunneling occurs when you stare at the same block of code for too long; your brain starts reading what you *meant* to write, rather than what is actually there.

Getting a fresh view—whether by explaining the problem to a colleague, talking to a rubber duck, or taking a walk—forces a reset in perspective.

”

A very simple but particularly useful technique is to explain the problem to someone else. You will often find the solution just in the act of explaining it.

— THE PRAGMATIC PROGRAMMER

The "Rubber Duck" approach:

```
// You explain: "The loop processes each item..."
for (int i = 0; i <= items.Count; i++)
{
    // "...wait. 'less than OR EQUAL'.
    // That's an IndexOutOfRangeException. Found it."
    ProcessItem(items[i]);
}
```


Pros & Cons

Pros

- Extremely fast way to break out of circular thinking.
- Articulating the problem often reveals the solution immediately.

Cons

- Requires stepping away, which feels counterproductive when rushed.
- Interrupts the flow state of teammates if you ask them for help.

Check the Plug

Why use this technique?

Developers often waste hours diving into complex logic and debugging correct code when the real issue is a broken environment or an incorrect setup.

Before diving into complex logic, verify your fundamental assumptions. Is the system actually running? Are you connected to the right database? "Checking the plug" prevents these wasted hours.

Before: Deep dive into complex logic

```
// Debugging for hours why the DB query fails
var results = _dbContext.Users
    .Where(u => u.IsActive)
    .ToList();
// "The query must be malformed. Let me rewrite the LINQ expression."
```

After: Checking fundamental assumptions

```
// 1. Verify connection first
// ping database.local

// 2. Check configuration variables
Console.WriteLine(_config["ConnectionStrings:DefaultDB"]);
// Output: null
// Ah. The appsettings.json connection string was missing.
```

Pros & Cons

Pros

- Saves days of effort tracking down non-existent code bugs.
- Builds a habit of verifying the testing environment first.

Cons

- Can feel insulting or too basic for experienced engineers.

END

Thank you!

Any questions?

